OS ORANGE PROBLEM

PES-VCS

Aman Rao U - PES2UG24CS054

Screenshots:

Phase 1:

1A)

```
amanraou@AOMEN-16:~/OS/PES2UG24CS054-pes-vcs$ make test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
amanraou@AOMEN-16:~/OS/PES2UG24CS054-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

1B)

```
amanraou@AOMEN-16:~/OS/PES2UG24CS054-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
```

Phase 2:

2A)

```
amanraou@AOMEN-16:~/OS/PES2UG24CS054-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

Phase 4:

4A)

```
amanraou@AOMEN-16:~/OS/PES2UG24CS054-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
amanraou@AOMEN-16:~/OS/PES2UG24CS054-pes-vcs$ echo "Hello" > hello.txt
amanraou@AOMEN-16:~/OS/PES2UG24CS054-pes-vcs$ ./pes add hello.txt
amanraou@AOMEN-16:~/OS/PES2UG24CS054-pes-vcs$ ./pes commit -m "Initial commit"
Committed: 6ccdf4e9c12f... Initial commit
amanraou@AOMEN-16:~/OS/PES2UG24CS054-pes-vcs$ echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"
Committed: f6d5b6f80d46... Add world
amanraou@AOMEN-16:~/OS/PES2UG24CS054-pes-vcs$ echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"
Committed: df11803c1e06... Add farewell
amanraou@AOMEN-16:~/OS/PES2UG24CS054-pes-vcs$ ./pes log
commit df11803c1e06f3a7420243de6277828a89f06a97e03c7e80b59e6f74ac159b62
Author: PES User <pes@localhost>
Date:   1776611886

    Add farewelldd fQ

commit f6d5b6f80d46185171553f400a6720758ffed79ec93d48d5a5cea85ed9a3f710
Author: PES User <pes@localhost>
Date:   1776611877

    Add worldelldd fQ

commit 6ccdf4e9c12f3bd22d75c74a6f3113c79374151a24320003a0989f03fca5c7d6
Author: PES User <pes@localhost>
Date:   1776611856

    Initial committial commiQ

amanraou@AOMEN-16:~/OS/PES2UG24CS054-pes-vcs$
```

4B)

```
amanraou@AOMEN-16:~/OS/PES2UG24CS054-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/6c/cdf4e9c12f3bd22d75c74a6f3113c79374151a24320003a0989f03fca5c7d6
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/df/11803c1e06f3a7420243de6277828a89f06a97e03c7e80b59e6f74ac159b62
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/f6/d5b6f80d46185171553f400a6720758ffed79ec93d48d5a5cea85ed9a3f710
.pes/refs/heads/main
```

4C)

```
amanraou@AOMEN-16:~/OS/PES2UG24CS054-pes-vcs$ cat .pes/refs/heads/main
df11803c1e06f3a7420243de6277828a89f06a97e03c7e80b59e6f74ac159b62
amanraou@AOMEN-16:~/OS/PES2UG24CS054-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
amanraou@AOMEN-16:~/OS/PES2UG24CS054-pes-vcs$
```

FINAL:

```
amanraou@AOMEN-16:~/OS/PES2UG24CS054-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
TEST_DIR=/tmp/tmp.VTAhQlRO7O
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 772cb963d2ed... Initial commit

Log after first commit:
commit 772cb963d2edc6c439166a51c43196f1bdfb7ad45f933a1123a84b91056657be
Author: Aman Rao U <PES2UG24CS054>
Date:   1776612570
```

```
    Initial committial cQ


--- Second Commit ---
Committed: a75d7b615838... Update file.txt

--- Third Commit ---
Committed: 220be5613f56... Add farewell

--- Full History ---
commit 220be5613f56f79b5d7c1114525ea7bd9430646d6b10414a52503313c2456689
Author: Aman Rao U <PES2UG24CS054>
Date:   1776612570

    Add farewellQ

commit a75d7b6158384f74359474b6069db4b08e50d8f9947a94de421698f8e5491654
Author: Aman Rao U <PES2UG24CS054>
Date:   1776612570

    Update file.txt

commit 772cb963d2edc6c439166a51c43196f1bdfb7ad45f933a1123a84b91056657be
Author: Aman Rao U <PES2UG24CS054>
Date:   1776612570

    Initial committial cQ


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
220be5613f56f79b5d7c1114525ea7bd9430646d6b10414a52503313c2456689

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
```

```
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/22/0be5613f56f79b5d7c1114525ea7bd9430646d6b10414a52503313c2456689
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/77/2cb963d2edc6c439166a51c43196f1bdfb7ad45f933a1123a84b91056657be
.pes/objects/a7/5d7b6158384f74359474b6069db4b08e50d8f9947a94de421698f8e5491654
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
```

PHASE 5 & 6:

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

To implement pes checkout , the file .pes/refs/heads/ is read to get the commit hash, and .pes/HEAD is updated to point to that branch. The working directory must then be updated to match the tree of that commit by recreating all files and directories from the object store. The index must also be updated accordingly. This operation is complex because it involves safely overwriting files, handling directory structures, and ensuring uncommitted changes are not lost.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

For each file in the index, compute the current hash from the working directory and compare it with the stored hash in the index. If they differ, the file has uncommitted changes. Then compare the index version with the version in the

target branch’s tree from the object store. If both differ, a conflict exists. In such cases, checkout must be refused to prevent overwriting user changes

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

In a detached HEAD state, new commits are created normally but are not referenced by any branch. HEAD moves forward, but no branch points to these commits, making them potentially unreachable later. A user can recover them by creating a new branch pointing to the current HEAD or by manually saving the commit hash in a branch reference.

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Start from all branch heads and add their commit hashes to a set of reachable objects. Traverse each commit to collect its parent and tree, and traverse trees to collect all referenced blobs and subtrees. Continue until all reachable objects are found. Then delete any objects in .pes/objects not present in this set. A hash set should be used for efficient lookup. For 100,000 commits, the total objects visited could be around 300,000 to 500,000 including commits, trees, and blobs.